

CSCI-GA.1170-001 Fundamental Algorithm

Problem Set 1

Varun Sahni
N10338688
NetID: vs3606

September 14, 2026

Problem 1-1 (Asymptotic Comparisons)

For each pair, compute $L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ and classify using the rule:

$$\begin{aligned} L = 0 &\implies O(g), o(g), \\ 0 < L < \infty &\implies O(g), \Theta(g), \Omega(g), \\ L = \infty &\implies \Omega(g), \omega(g). \end{aligned}$$

- (a) $f(n) = 3n^8 + 20n^6$, $g(n) = 2n^8 - 300n^6$.

Divide the numerator and denominator by the dominant term n^8 :

$$L = \lim_{n \rightarrow \infty} \frac{3n^8 + 20n^6}{2n^8 - 300n^6} = \lim_{n \rightarrow \infty} \frac{3 + \frac{20}{n^2}}{2 - \frac{300}{n^2}} = \frac{3}{2}.$$

Since $0 < L < \infty$, f is $O(g)$, $\Theta(g)$, and $\Omega(g)$.

- (b) $f(n) = (\log_{10} n)^3$, $g(n) = \log_2(n^3) + \log_{10} n$.

Convert every logarithm to the natural logarithm $\ln n$:

$$f(n) = \left(\frac{\ln n}{\ln 10} \right)^3 = \frac{(\ln n)^3}{(\ln 10)^3}, \quad g(n) = \frac{3 \ln n}{\ln 2} + \frac{\ln n}{\ln 10} = \ln n \left(\frac{3}{\ln 2} + \frac{1}{\ln 10} \right).$$

Therefore

$$L = \lim_{n \rightarrow \infty} \frac{\frac{(\ln n)^3}{(\ln 10)^3}}{\ln n \left(\frac{3}{\ln 2} + \frac{1}{\ln 10} \right)} = \lim_{n \rightarrow \infty} \frac{(\ln n)^2}{(\ln 10)^3 \left(\frac{3}{\ln 2} + \frac{1}{\ln 10} \right)} = \infty,$$

since $(\ln n)^2 \rightarrow \infty$ while the denominator is a positive constant.

Since $L = \infty$, f is $\Omega(g)$ and $\omega(g)$.

- (c) $f(n) = n^n$, $g(n) = 2^{n^2}$.

Rewrite $f(n)$ with base 2 using $n = 2^{\log_2 n}$:

$$f(n) = n^n = (2^{\log_2 n})^n = 2^{n \log_2 n}.$$

Then, using $\frac{2^a}{2^b} = 2^{a-b}$:

$$L = \lim_{n \rightarrow \infty} \frac{2^{n \log_2 n}}{2^{n^2}} = \lim_{n \rightarrow \infty} 2^{n \log_2 n - n^2}.$$

Factor the exponent: $n \log_2 n - n^2 = n(\log_2 n - n)$.

Since n grows faster than $\log_2 n$, the bracket $\log_2 n - n \rightarrow -\infty$; multiplying by the growing factor n keeps it at $-\infty$:

$$n(\log_2 n - n) \rightarrow -\infty.$$

Hence the exponent tends to $-\infty$, and $2^x \rightarrow 0$ as $x \rightarrow -\infty$:

$$L = \lim_{n \rightarrow \infty} 2^{n(\log_2 n - n)} = 0.$$

Since $L = 0$, f is $O(g)$ and $o(g)$.

- (d) $f(n) = n^2 \log_2 n$, $g(n) = n(\log_2 n)^5$.

Cancel the common factor $\log_2 n$:

$$L = \lim_{n \rightarrow \infty} \frac{n^2 \log_2 n}{n(\log_2 n)^5} = \lim_{n \rightarrow \infty} \frac{n}{(\log_2 n)^4}.$$

To evaluate this limit, substitute $m = \log_2 n$, so that $n = 2^m$ and $m \rightarrow \infty$ as $n \rightarrow \infty$:

$$L = \lim_{m \rightarrow \infty} \frac{2^m}{m^4} = \infty,$$

because the exponential 2^m grows faster than any polynomial m^4 .

Since $L = \infty$, f is $\Omega(g)$ and $\omega(g)$.

- (e) $f(n) = 25^n(n + \log_2 n)^3$, $g(n) = 5^{2n}\sqrt{n^6 - n^5}$.

Note that $5^{2n} = (5^2)^n = 25^n$, so this common factor cancels. Factor out the dominant term n^3 :

$$f(n) = 25^n n^3 \left(1 + \frac{\log_2 n}{n}\right)^3, \quad g(n) = 25^n \sqrt{n^6 \left(1 - \frac{1}{n}\right)} = 25^n n^3 \sqrt{1 - \frac{1}{n}}.$$

Therefore

$$L = \lim_{n \rightarrow \infty} \frac{\left(1 + \frac{\log_2 n}{n}\right)^3}{\sqrt{1 - \frac{1}{n}}} = \frac{(1+0)^3}{\sqrt{1-0}} = 1.$$

Since $0 < L < \infty$, f is $O(g)$, $\Theta(g)$, and $\Omega(g)$.

- (f) $f(n) = \log_2 n$, $g(n) = n^\epsilon$ for any $\epsilon > 0$.

Convert to the natural logarithm:

$$L = \lim_{n \rightarrow \infty} \frac{\log_2 n}{n^\epsilon} = \lim_{n \rightarrow \infty} \frac{\frac{\ln n}{\ln 2}}{n^\epsilon} = \frac{1}{\ln 2} \lim_{n \rightarrow \infty} \frac{\ln n}{n^\epsilon}.$$

The remaining limit is of the indeterminate form $\frac{\infty}{\infty}$, so apply L'Hôpital's rule:

$$\lim_{n \rightarrow \infty} \frac{\ln n}{n^\epsilon} = \lim_{n \rightarrow \infty} \frac{\frac{d}{dn}(\ln n)}{\frac{d}{dn}(n^\epsilon)} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{\epsilon n^{\epsilon-1}} = \lim_{n \rightarrow \infty} \frac{1}{\epsilon n^\epsilon} = 0.$$

Hence $L = 0$ for every $\epsilon > 0$.

Since $L = 0$, f is $O(g)$ and $o(g)$.

Problem 1-2 (Order of Growth)

1. $f_1(n) = 3^n(\log_3(n^2))^5$.

First simplify the log: $\log_3(n^2) = 2 \log_3 n$.

Then $f_1(n) = 3^n(2 \log_3 n)^5 = 3^n \cdot 32 (\log_3 n)^5 = \Theta(3^n \log^5 n)$.

2. $f_2(n) = (n \log_2(n^2))^3$.
First simplify the log: $\log_2(n^2) = 2 \log_2 n$.
Then $f_2(n) = (n \cdot 2 \log_2 n)^3 = (2n \log_2 n)^3 = 8 n^3 \log_2^3 n = \Theta(n^3 \log^3 n)$.
3. $f_3(n) = 2^{2n} \log_2(n^2 + n)$.
First note $2^{2n} = (2^2)^n = 4^n$.
Also $\log_2(n^2 + n) = \Theta(\log_2 n^2) = \Theta(\log n)$.
Thus $f_3(n) = 4^n \cdot \Theta(\log n) = \Theta(4^n \log n)$.
4. $f_4(n) = n^2 \sqrt{n} + 1$.
Since $\sqrt{n} = n^{1/2}$, we have $n^2 \cdot n^{1/2} = n^{2+1/2} = n^{5/2}$.
Thus $f_4(n) = n^{5/2} + 1 = \Theta(n^{5/2})$.
5. $f_5(n) = \sqrt{\log_4(n^4)}$.
First simplify the log: $\log_4(n^4) = 4 \log_4 n$.
So $f_5(n) = \sqrt{4 \log_4 n} = 2 \sqrt{\log_4 n}$.
Since $\log_4 n = \frac{\ln n}{\ln 4}$, we have $\sqrt{\log_4 n} = \Theta(\sqrt{\log n})$.
Thus $f_5(n) = \Theta(\sqrt{\log n})$.
6. $f_6(n) = n^{\frac{n}{\log_2 n}} \log_{n^4}(2n)$.
First, using $n = 2^{\log_2 n}$: $n^{\frac{n}{\log_2 n}} = (2^{\log_2 n})^{\frac{n}{\log_2 n}} = 2^n$.
Next, by change of base: $\log_{n^4}(2n) = \frac{\log_2(2n)}{\log_2(n^4)} = \frac{1 + \log_2 n}{4 \log_2 n} = \Theta(1)$.
Thus $f_6(n) = 2^n \cdot \Theta(1) = \Theta(2^n)$.
7. $f_7(n) = n + \log_n 4$.
By change of base: $\log_n 4 = \frac{\log_2 4}{\log_2 n} = \frac{2}{\log_2 n}$.
Thus $f_7(n) = n + \frac{2}{\log_2 n} = \Theta(n)$.
8. $f_8(n) = n \log_2(n!)$.

Upper bound: since every factor of $n!$ is at most n , we have $n! \leq n^n$, so

$$\log_2(n!) \leq \log_2(n^n) = n \log_2 n.$$

Lower bound: among the n factors $1, 2, \dots, n$, at least half of them are $\geq \frac{n}{2}$. Dropping the smaller half gives

$$n! \geq \left(\frac{n}{2}\right)^{n/2},$$

and therefore

$$\log_2(n!) \geq \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \frac{n}{2} \log_2 n - \frac{n}{2}.$$

The upper bound is $O(n \log n)$, and the lower bound is $\Omega(n \log n)$ because the term $\frac{n}{2} \log_2 n$ dominates the subtractive term $\frac{n}{2}$. Together,

$$\log_2(n!) = \Theta(n \log n).$$

Thus $f_8(n) = n \cdot \Theta(n \log n) = \Theta(n^2 \log n)$.

(b) Asymptotically increasing order (smallest to largest):

$$\Theta(\sqrt{\log n}) \subset \Theta(n) \subset \Theta(n^2 \log n) \subset \Theta(n^{5/2}) \subset \Theta(n^3 \log^3 n) \subset \Theta(2^n) \subset \Theta(3^n \log^5 n) \subset \Theta(4^n \log n).$$

Justification: any power of n dominates any power of $\log n$ ($n > \log^c n$ for every c), so $\sqrt{\log n} < n$; among polynomial terms, a larger exponent dominates ($n < n^2 \log n < n^{5/2} < n^3 \log^3 n$); any exponential dominates

any polynomial ($2^n > n^3 \log^3 n$); and among exponentials a larger base dominates ($2^n < 3^n \log^5 n < 4^n \log n$, since the base decides the order regardless of the logarithmic factor).

Problem 1-3 (Counting Points)

- (a) Naive algorithm running time.

For each interval $I[i]$, the naive approach scans all n points and checks whether each point lies inside the interval:

$$\underbrace{\text{one loop over } n \text{ intervals}}_{\Theta(n)} \times \underbrace{\text{one loop over } n \text{ points}}_{\Theta(n)} = \Theta(n^2).$$

- (b) Build an array C of $3n$ records.

Each point and each endpoint becomes a record with three fields: a numeric **value**, a **type** (“point”, “left”, or “right”), and a **pointer** back to the point/interval index. With n points and $2n$ endpoints, C holds $3n$ records:

```
GENERATEC(X, I, n):
  Let C be a new array of size 3n
  For i = 1 to n:
    C[3i-2].value = X[i]
    C[3i-2].type = "point"
    C[3i-2].pointer = i
    C[3i-1].value = I[i].left
    C[3i-1].type = "left"
    C[3i-1].pointer = i
    C[3i].value = I[i].right
    C[3i].type = "right"
    C[3i].pointer = i
  Return C
```

- (c) Sort C by value, then sweep and accumulate a running counter.

After sorting, process the records in increasing order of **value** while keeping a single running counter. This uses only $O(1)$ additional memory beyond the output array K :

```
COMPUTEK(C, n):
  Let K be a new array of size n initialized to 0
  counter = 0
  For k = 1 to 3n:
    If C[k].type == "point":
      counter = counter + 1
    Else If C[k].type == "left":
      K[C[k].pointer] = K[C[k].pointer] - counter
    Else If C[k].type == "right":
      K[C[k].pointer] = K[C[k].pointer] + counter
  Return K
```

- (d) Running time of each step:

- Generating C : a single loop over n with constant work per entry, so $\Theta(n)$.
- Sorting C : a comparison sort of $3n$ records, so $\Theta(3n \log(3n)) = \Theta(n \log n)$.
- Computing K : a single sweep over $3n$ records, so $\Theta(n)$.

Overall running time:

$$\Theta(n) + \Theta(n \log n) + \Theta(n) = \Theta(n \log n).$$

(e) Loop invariant and proof of correctness.

Invariant: at the start of the k -th iteration, **counter** equals the number of points in X strictly less than $C[k].value$ (assuming no collisions). Moreover, for each segment i , the value $K[i]$ is in the correct phase:

- if $I[i].left$ has not yet been processed, $K[i] = 0$;
- if the sweep is strictly between $I[i].left$ and $I[i].right$, $K[i] = -(\text{number of points} < I[i].left)$;
- if $I[i].right$ has already been processed, $K[i]$ equals the number of points contained in $I[i]$.

Initialization: before $k = 1$, **counter** = 0 and every $K[i] = 0$; there are no points smaller than the first value, and no endpoints have been processed, so the invariant holds.

Maintenance: assume the invariant holds before iteration k .

- If $C[k]$ is a “point”, then **counter** increments, correctly tracking all points seen so far; K is unchanged.
- If $C[k]$ is a “left” endpoint of segment i , then $K[i]$ is set to $-\text{counter}$, i.e. $-(\text{number of points} < I[i].left)$.
- If $C[k]$ is a “right” endpoint of segment i , then $K[i]$ is increased by **counter**, the number of points $\leq I[i].right$.
Combined with the stored value, this yields

$$K[i] = (\text{points} \leq I[i].right) - (\text{points} < I[i].left).$$

In every case the invariant is re-established for the next iteration.

Termination: after $k = 3n$, all records have been processed, in particular every “right” endpoint. Hence for every segment i ,

$$K[i] = (\text{points} \leq I[i].right) - (\text{points} < I[i].left),$$

which is exactly the number of points contained in the segment $I[i]$.

(f) Collision counterexample.

Let $n = 1$, $X[1] = 1$, $I[1].left = 1$, $I[1].right = 1$.

Assume the sort breaks ties in the order “point” < “left” < “right”.

Then sorted $C = [(1, \text{“point”}, 1), (1, \text{“left”}, 1), (1, \text{“right”}, 1)]$:

- $k = 1$ (“point”): **counter** becomes 1.
- $k = 2$ (“left”): $K[1] = 0 - 1 = -1$.
- $k = 3$ (“right”): $K[1] = -1 + 1 = 0$.

The output is $K[1] = 0$, but point 1 lies in segment $[1, 1]$, so the correct count is 1: the algorithm is incorrect under this tie-breaking.

(g) Fix: separate coincident coordinates by mapping.

The failure in (f) occurs because the point at coordinate 1 is processed *before* the left endpoint at the same coordinate, so it is never counted. To make the algorithm correct regardless of tie-breaking, map

each coordinate to a value that encodes the desired processing order “left < point < right” at every coordinate:

$$\text{point } x \mapsto 3x, \quad \text{left endpoint } x \mapsto 3x - 1, \quad \text{right endpoint } x \mapsto 3x + 1.$$

At any common coordinate v these become $3v - 1 < 3v < 3v + 1$, so left endpoints are processed first, then points, then right endpoints, independent of the sort’s tie-breaking:

```

GENERATEC_MODIFIED(X, I, n):
  Let C be a new array of size 3n
  For i = 1 to n:
    C[3i-2].value = 3 * X[i]
    C[3i-2].type = "point"
    C[3i-2].pointer = i
    C[3i-1].value = 3 * I[i].left - 1
    C[3i-1].type = "left"
    C[3i-1].pointer = i
    C[3i].value = 3 * I[i].right + 1
    C[3i].type = "right"
    C[3i].pointer = i
  Return C

```

Verification on the example in (f): the point maps to 3, the left endpoint to 2, and the right endpoint to 4. Processing in order left (2) → point (3) → right (4) gives $K[1] = 0$ at the left endpoint (counter = 0), then **counter** becomes 1 at the point, and finally $K[1] = 0 + 1 = 1$ at the right endpoint, which is correct.